

Coin-Detection-using-OpenCV-in-Python

Name : DINESH R

Reg.No:212224240037

Aim:

To develop an AI-based image processing system that can automatically detect and count coins in an image using Python and OpenCV, while visualizing all the intermediate processing steps such as grayscale conversion, blurring, edge detection, and contour detection.

OBJECTIVE:

1. To apply fundamental computer vision techniques to identify circular objects (coins).
2. To understand the use of image preprocessing and feature extraction using OpenCV.
3. To display all intermediate outputs to explain how detection is achieved.
4. To count and label the number of coins accurately.

ALGORITHM:

1. Start
2. Input the image (coins image file).

3. Convert the image to grayscale to simplify analysis.
4. Apply Gaussian Blur to reduce image noise and smooth edges.
5. Apply Canny Edge Detection to find edges of coins.
6. Find Contours in the edge-detected image.
7. Filter Contours based on area (to remove small noise).
8. Draw circles around detected coins and assign serial numbers.
9. Count the total number of coins detected.
10. End.

Program:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os
image_path = "coin.jpg"
# Load image
image = cv2.imread(image_path)
if image is None:
    raise FileNotFoundError(f"Image not found at {image_path}")
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (11, 11), 0)
edges = cv2.Canny(blurred, 30, 150)
contours, _ = cv2.findContours(edges.copy(), cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
output = image.copy()

coin_count = 0
```

```

for contour in contours:
    # Filter out small contours (noise)
    area = cv2.contourArea(contour)
    if area < 300:
        continue

    # Draw contour
    ((x, y), radius) = cv2.minEnclosingCircle(contour)
    if radius > 15:
        coin_count += 1
        cv2.circle(output, (int(x), int(y)), int(radius), (0, 255, 0), 2)
        cv2.putText(output, f"{coin_count}", (int(x - 10), int(y + 10)),
                     cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)

print(f"Total coins detected: {coin_count}")
Total coins detected: 1
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.title("Original Image")
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.subplot(1, 2, 2)
plt.title(f"Detected Coins: {coin_count}")
plt.imshow(cv2.cvtColor(output, cv2.COLOR_BGR2RGB))
plt.axis("off")

plt.show()

fig, axs = plt.subplots(1, 5, figsize=(20, 6))

axs[0].imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
axs[0].set_title("Original Image")
axs[0].axis("off")

axs[1].imshow(gray, cmap='gray')
axs[1].set_title("Grayscale Image")

```

```
axs[1].axis("off")

axs[2].imshow(blurred, cmap='gray')
axs[2].set_title("Blurred Image")
axs[2].axis("off")

axs[3].imshow(edges, cmap='gray')
axs[3].set_title("Edge Detection (Canny)")
axs[3].axis("off")

axs[4].imshow(cv2.cvtColor(output, cv2.COLOR_BGR2RGB))
axs[4].set_title(f"Detected Coins: {coin_count}")
axs[4].axis("off")

plt.tight_layout()
plt.show()
```

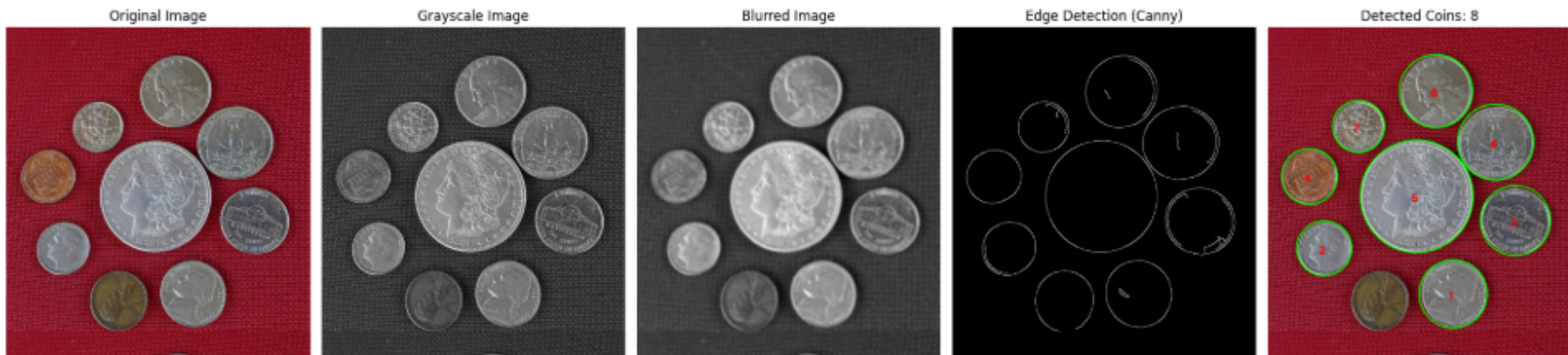
Output:

Original Image



Detected Coins: 8





Result:

The system successfully detected and counted all coins in the given image.